Journal of Parallel and Distributed Computing 201 (2025) 105071

Contents lists available at ScienceDirect

Journal of Parallel and Distributed Computing

journal homepage: www.elsevier.com/locate/jpdc

Latency and cost-aware consumer group autoscaling in message broker systems

Diogo Landau [a], Nishant Saurabh [a,*], Xavier Andrade [b], Jorge G. Barbosa [c,*]

[a] Department of Information and Computing Sciences, Utrecht University, Netherlands
[b] Departamento de Engenharia e Gestão Industrial, Universidade do Porto - FEUP, Portugal
[c] LIACC, Departamento de Engenharia Informática, Faculdade de Engenharia, Universidade do Porto, Portugal

ARTICLE INFO

Keywords:
Variable item size
Bin packing
Consumer group autoscaling
Message broker

ABSTRACT

Message brokers often facilitate communication between data producers and consumers by adding variable-sized messages to ordered distributed queues. Our goal is to determine the number of consumers and consumer partition assignments needed to ensure that the data consumption rate matches the data production rate. We model this problem as a variable item size bin packing problem. As the production rate varies, new consumer–partition assignments are computed, potentially requiring the reallocation of partitions from one consumer to another. During reallocation, data in the queue are not read, leading to increased latency costs. To address this problem, we focus on the multiobjective optimization cost of minimizing the number of consumers and reducing latency. We introduce several heuristic algorithms and compare them to state-of-the-art heuristics. In our experimental setup, the proposed modified worst fit (MWF) heuristic achieves a 48% reduction, with a similar number of consumers, in comparison with the best fit decrease (BFD). In addition, MWF achieves a 99th percentile latency of 2.24 seconds compared with that of 364.66 with the approach by Kafka using the same number of consumers. Alternatively, to obtain a lower 99th percentile latency than our approach does, Kafka requires at least 60% more consumers than our method requires.

1. Introduction

Brokers are common employed to facilitate communication in distributed environments [1]. These systems enable asynchronous communication between producers and consumers, addressing some of the challenges that are common within distributed and concurrent data processing [2]. In this context, applications that require real-time data processing, such as those presented in [3] and [4], often require estimating the required number of consumers that can handle the expected application load. However, these methods overestimate the resources required to minimize the application's latency, and the expected load patterns are highly unpredictable. As such, a dynamic consumer group autoscaling solution that scales the set of consumers on the basis of the current demand while simultaneously minimizing the overall latency is needed.

Unfortunately, consumer group autoscaling solutions often face a broad set of challenges related to ordered message delivery, skewed data distributions, production rate variability, and rebalancing costs. Ordered message delivery provides applications such as financial transaction processing systems [5] with the required guarantees that the system will eventually reach its expected state. The *queue/partition* is a message broker component that is responsible for delivering messages to consumers in the same order as they were produced, as long as they are assigned to only one consumer. Given that data that have to be delivered in a particular order have to be sent to the same queue, two distinct queues will frequently have different volumes of data (skewed data distributions) and consequently different rates of production [6]. Furthermore, a queue's production rate varies over time, which is a natural consequence of most serving applications with loads that vary with the current demand [7]. This variability is one of the main driving forces behind a consumer group autoscaling solution. Consider a scenario where a consumer has been assigned a set of partitions, and the combined production rate gradually increases to the point where the consumer can no longer keep up with the load. If these partitions are

* Corresponding author.
E-mail addresses: d.landau@uu.nl (D. Landau), n.saurabh@uu.nl (N. Saurabh), xavier.andrade@fe.up.pt (X. Andrade), jbarbosa@fe.up.pt (J.G. Barbosa).

https://doi.org/10.1016/j.jpdc.2025.105071
Received 29 July 2024; Received in revised form 9 February 2025; Accepted 26 March 2025
Available online 28 March 2025
0743-7315/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

not reassigned to account for this excess load, the message latency will increase indefinitely, which will in turn delay processing [8]. Rebalancing a partition to another consumer, however, introduces another cost that is related to the fact that, in that period of time, no consumer is reading data from it, which will have an impact on latency [3,9]. However, the smaller the rebalancing time is, the less latency is experienced.

Chindanonda et al. [9] attempted to forecast the expected production rate and, on the basis of this estimate, determine the number of consumers required to process the load. Other approaches, such as the Kubernetes event-driven architecture, have leveraged consumer group statistics such as message latency to incrementally add members to a group to increase consumption capacity. However, neither approach considers skewed data distributions. Instead, the number of partitions is simply divided by the members in the consumer group. As shown in our experiment, this approach very often leads to over allocating a consumer, which leads to increased latency. Ezzeddine et al. [10] modeled this problem for a case where the messages in a queue are constant in size. In their most recent work [8], they proposed a synchronous blocking reassignment procedure that interrupts all partition consumption throughout the whole rebalancing procedure. To address the reassignment overhead by approaches such as [8], [3] proposed an incremental reassignment strategy that only interrupts the consumption for rebalanced partitions while also minimizing the partitions that are in fact rebalanced. However, [3] did not consider the problem of minimizing the number of consumers in a group.

In this work, we model consumer group autoscaling as a variation of the bin packing (BP) problem by considering consumers as bins and partitioning items that have to be assigned a bin. Since the rate at which data are inserted into a queue varies over time, a BP item's size also varies with time. In fact, a queue's size correlates with its current write speed, which fluctuates on the basis of the current system's load. This inevitably implies that a BP solution for a given time instant may not hold true in future instants. Owing to this BP variation, a new solution must be computed at different time instants, each having different information concerning the size of each item (each queue's write speed for that time instant). This might lead to a queue being reallocated to another consumer (bin). Given the overhead induced by the rebalancing procedure, there is an additional cost to consider related to latency [11]. As such, to minimize the operational and rebalancing costs simultaneously, we propose a framework based on four variations of BP heuristic algorithms, three of which are described in Section 6.4 to be a competitive alternative to the multiobjective optimization problem at hand.

To summarize, the primary contributions of this paper are as follows:

- Modelling the consumer group autoscaling problem to consider variable message sizes and production rates for generalized message broker use;
- Defining the Rscore metric to quantify latency between two consecutive consumer group assignments;
- Exploring four variations of the bin packing approximation algorithms;
- Implementing a fully functional framework for the consumer group autoscaling problem;
- Evaluating the autoscaler implementation's response time when autoscaling the consumer group in a *Kafka* production infrastructure.

In Section 2, we introduce the background, outlining the problem domain and existing bin packing heuristics. Section 3 presents previous research on consumer group autoscaling and applications of the bin packing problem. In Section 4 the problem is formally defined, and in Section 5, the proposed approach is introduced. Section 6 presents a comparison through simulation of the proposed and state-of-the-art algorithms. Section 7 presents an implementation of a consumer group autoscaling system for Kafka. Section 8 presents an empirical evaluation of a production Kafka platform, and finally, Section 9 concludes the paper.

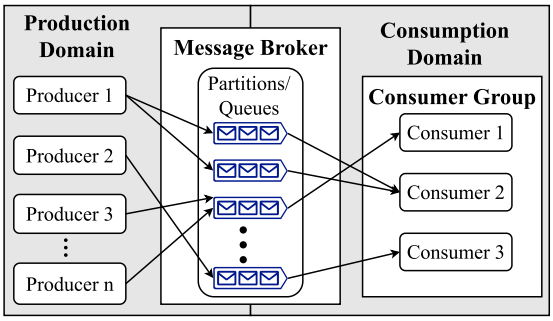


Fig. 1. Representation of the data production and consumption domains within a message broker environment.

2. Background

A group of consumers consumes data from several partitions/queues with varying loads within the message broker. The set of partitions is assigned to different consumers in the group to parallelize and distribute data consumption. This problem is depicted in the data consumption domain of Fig. 1.

The aim is to establish a deterministic approach for determining the required number of consumers operating in parallel. This approach ensures that the production rate across a set of partitions is not higher than the consumption rate by a consumer group while minimizing the operational cost. Moreover, the system is dynamic, and the reassignment of consumers to partitions needs to be performed when consumer overload is detected. Otherwise, messages would accumulate, leading to lag between the last message inserted and the last message read by the consumer group, increasing with time. Additionally, a single partition can only be assigned to a single consumer to ensure that the messages are consumed in the same sequence as they were published. The assignment of partitions to consumers can be modeled as a bin packing problem; therefore, an overview of the bin packing formulation is given next.

The classical bin packing (BP) problem considers an infinite supply of bins with a capacity C and a list L of items with sizes no larger than C [12]. The problem consists of packing the set of items into a minimum number of bins so that the size of the items in each bin is no greater than C.

The BP problem has been extensively reviewed in the literature [13–15]. Given the NP-hard nature of BP and the real-time constraints of autoscaling a consumer group, this study focuses on approximation algorithms that provide heuristic solutions to the problem in low-order polynomial time, as opposed to the higher time complexity of other heuristics, such as the minimum bin slack (MBS) [16][17] or variable neighborhood search methods [18][19].

A method is presented in [12] to classify approximation algorithms that solve the BP problem and are used throughout this paper. During an algorithm's execution, a bin is either *open* or *closed*. In the former, the bin can still be used to add additional items, whereas in the latter, it is no longer available. The list of bins is indexed from left to right, and they are filled in that exact order. The number of bins used by an algorithm can be computed via the index of the first empty bin in the bin list. When requesting or creating a bin, the process involves opening the lowest indexed empty bin (i.e., the leftmost empty bin).

Theoretically, a significant amount of research has been conducted to estimate the worst-case performance of approximation algorithms. Using $A(L)$ to denote the number of bins, a certain algorithm makes use of a configuration of items L. Let $OPT(L)$ represent the number of bins required to achieve the optimal solution, and let Ω be the set of all possible lists, each with a different arrangement of their items. $R_A(k)$ (Equation (1)) serves as a performance metric relative to the optimal number of bins used by an algorithm.

$$R_A(k) = \sup_{L\in\Omega}\left\{\frac{A(L)}{k} : k = OPT(L)\right\} \quad (1)$$

The asymptotic performance ratio (APR) of an algorithm A, R_A^∞, is defined by Equation (2) and represents the worst-case performance result. This metric is commonly used to measure the packing quality of BP approximation algorithms.

$$R_A^\infty = \limsup_{k\to\infty} R_A(k) \quad (2)$$

In the context of the consumer group autoscaling problem, the input is a list of the production rates for all queues. For the first assignment, offline any fit (AF) heuristics can perform queue-consumer mapping, as all item sizes are known beforehand. When rebalancing, the aim is to reduce the cost of rescheduling in terms of latency, and therefore, a variation of semi-online AF approximation algorithms is proposed to minimize the impact of item reallocation.

We extend the any fit algorithms in Section 5 as part of our methodology and adapt offline BP algorithms for this multiobjective context. These adapted algorithms serve as a baseline for comparing the performance of our proposed algorithm in Section 6.

An algorithm belonging to the AF class of algorithms must satisfy the following conditions [12]: *if bin j is empty, an item will not be assigned to it if it can fit into any bin to the left of j*.

The AF class of algorithms performs best if the list of items is sorted in decreasing order prior to assigning items to the bins. The following offline approximation algorithms apply this sorting strategy, except for the next fit decreasing (NFD) algorithm, which belongs to the aforementioned class. The first fit decreasing (FFD) heuristic places each item in the leftmost bin that can accommodate it without exceeding the bin's capacity. As shown in [20], this algorithm's APR is $R_{FFD}^\infty \approx 1.222$.

For each item, the worst fit decreasing (WFD) heuristic attempts to assign an item to the existing bin with the most available space. If the size constraint is not met, a new bin is created and assigned. This algorithm has as APR $R_{WFD}^\infty \approx 1.6910$. The best fit decreasing (BFD) algorithm follows a similar approach but places the item in the bin where it fits most tightly. If no suitable bin is available, a new bin is created and the item is inserted [21]. The BFD heuristic has an APR of $R_{BFD}^\infty \approx 1.222$. Finally, the NFD algorithm has only a single open bin at a time, which is always the most recently created bin. This algorithm attempts to place the item in the rightmost nonempty bin, and if it does not fit, then the bin to the right is used. As shown in [22], this algorithm has as APR $R_{NFD}^\infty \approx 1.6910$.

3. Related work

This section explored other contexts in which the bin packing model has been applied, demonstrating its effectiveness when representing a minimization and assignment problem similar to the one being studied. We then present the state-of-the-art in consumer group autoscaling, followed by a short summary of the related work in the context of our research.

3.1. Bin packing applications

There are several applications where BP is used to provide a suboptimal solution, such as operator placement for distributed data streams processing [23], the industrial cutting optimization problem [24,25], the temporal bin packing problem [26], throughput maximization in mobile cloudlets [27], elastic and scalable data streaming systems [28], complex event processing [29], and finally, related to cloud computing, the virtual machine placement (VMP) problem.

The VMP problem has gained increasing attention due to the growing use of cloud providers to support companies' technological infrastructure. This problem can be generalized to a set of virtual machines (VM), each assigned one of the cloud provider's physical machines (PMs) while attempting to minimize the operational cost (number of PMs used). As new VM requests arrive, each new item (VM) must be assigned a bin (PM). Furthermore, an item that has been previously assigned can be migrated to another physical machine since migration is made available via virtualization technology. Therefore, VMP is considered a fully dynamic bin packing problem, where items arrive and depart over time (online bin packing) and migrations are permitted [30–33]. Moreover, the migration cost at each decision interval is measured as the number of migrations [31] or computed on the basis of the size of all VMs that must be migrated [32].

Song et al. [33] and Kamali et al. [30] investigated a case where the items (VMs) require a variable amount of resources throughout their lifetime; therefore, the item size varies over time. This formulation is similar to the problem we are studying in the consumer group autoscaling context: the variable item size bin packing (VISBP) problem. Nevertheless, the algorithms proposed in [30] and [33] differ from ours since they consider that there are different types of items and that a bin should have a mixture of items so as not to overload the server (bin), allowing moderate item size changes (VMs).

3.2. Consumer group autoscaling

To comply with latency service level agreements in a publish–subscribe message broker environment, it is common to enable consumer group parallelism to distribute the load between the elements of a group [34–36]. As such, when the rate of production increases and the consumer group falls behind, it is expected that the group of consumers scales up and that partitions are reassigned.

Currently, Kubernetes' horizontal pod autoscaler (HPA)[1] paired with Kubernetes' event-driven architecture (KEDA) [2] enables consumer group autoscaling on the basis of the average CPU usage, memory, and event lag among group members. These threshold-based techniques scale the consumer group incrementally, depending on the group's current average performance with respect to the chosen strategy. However, Wang [37] showed that these threshold-based techniques are not accurate, resulting in resource overprovisioning.

Chindanonda et al. [9] developed a strategy to forecast the rate of production, enabling the computation of the estimated number of consumers required in the consumer group. Given a set of consumers, the threshold-based strategies (e.g., HPA and KEDA) and the algorithm presented in [9] distribute the number of partitions evenly between the consumers. This is also the default strategy used by the state-of-the-art message broker *Kafka*. This strategy fails to load balance between different consumers when the workload presents a skewed distribution, since the production into each event queue is independent and not necessarily equal.

Ezzeddine et al. [10] highlighted the skewed distribution problem and proposed monitoring consumer group lag relative to each queue to track the event distribution. Given a consumer's capacity and the queue lag, the authors assigned partitions to consumers using the first fit decreasing bin packing algorithm. Although the authors introduced a load-aware partition–consumer assignment, using the lag as a metric to determine each queue's production rate is inaccurate when the messages are published in each queue have variable sizes. Once the consumption rate is limited by the network throughput, to improve a queue's load model, production and consumption rates should be measured in bytes per second ($bytes/s$). As we study the case where messages have variable sizes, the approach introduced by [10] is not directly applicable to our problem. Therefore, we modify their approach by measuring the production and consumption rates in $bytes/s$ and use it to compare with those of our proposed algorithms.

The cost of rebalancing partitions between consumers due to data consumption downtime from partition rebalancing is highlighted in [9].

[1] https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.

[2] https://keda.sh/docs/2.9/scalers/apache-kafka/.

Despite this fact, the proposed rebalancing strategy involved imposing a time constraint for when the group could scale based on its last scaling decision. With respect to [10], the authors did not consider the rebalancing cost.

Blee-Goldman [3] introduced a cooperative incremental partition rebalancing protocol for Kafka, without interrupting the consumption of the partitions that remain assigned to the same consumer. Their aim was to load balance the data production between the consumers in a group and prioritize an existing partition assignment to reduce rebalancing time. The limitation of this solution is that consumer group minimization is not performed. Partitions are simply redistributed when consumers are added or removed from a group. Another limitation is that rebalancing only occurs when the group is scaled up or down, i.e., it is not capable of identifying when a rebalancing operation is needed.

More recently, Ezzeddine et al. [8] proposed a latency-aware and resource-efficient system that is based on a bin packing formulation. However, unlike the incremental approach proposed by [3], their method employs a blocking synchronization protocol that interrupts data consumption, discards previous assignments, and reassigns the partitions to the members of the consumer group. As indicated by [3], nonincremental partition rebalancing has drawbacks: rebalancing duration scales with the numbers of partitions and consumers, and no member of the group performs any work while partitions are reassigned. Like [3], our approach leverages the benefits of an incremental reassignment strategy and improves on it by simultaneously minimizing the operational and rebalancing costs.

3.3. Summary

The state-of-the-art consumer group autoscaling methods focus on solving distributed queue scenarios, which present limitations in real-world production environments. The first common assumption is that the production rate in all distributed queues is equal (attempting to assign the same number of partitions to different consumers in a group). The second assumption is that all messages produced in different event queues have the same size (measuring production and consumption rates in terms of the number of events per second). Existing solutions lack a deterministic metric to consider the rebalancing cost and introduce conservative heuristic algorithms to migrate the items to different bins (in some cases, items with higher costs are not migrated).

The formulation proposed in this paper overcomes these limitations by considering skewed distributions in the queue load and variable message sizes by considering $bytes/s$ instead of $events/s$. In addition, the proposed approach does not require a synchronization barrier for performing rebalancing and simultaneously minimizes the operational and rebalancing costs.

4. Problem formulation

This section presents the formulations of the assumptions and models underlying our approach to solve the consumer group autoscaling problem. Given the repeated use of symbols across multiple sections of the paper, Table 1 provides a summary of each symbol and its accompanying description. The symbols that have not yet been discussed will be presented throughout the paper.

4.1. Operational cost

We model the problem as the single-bin-size bin packing problem, where the bins are consumers whose capacity is the maximum consumption rate and the items are the partitions and their respective writing rates. The problem, then, is to find the minimum amount of consumers to assign all the partitions and ensure that the sum of the writing rates of the partitions assigned to a single consumer does not exceed its maximum consumption capacity C.

Table 1
Index of the symbols referenced in multiple sections of the paper.

Symbol	Description
R_A^∞	Bin packing algorithm A's asymptotic performance ratio.
C	Maximum bin size for the Bin Packing algorithm. Also used to represent the consumer's maximum throughput.
P	Set of partitions data has to be read from.
B	Set of consumers.
$\mathbf{X}(t)$	Paritition-consumer assignment matrix at time t.
k	Iteration during which an assignment matrix is computed.
t_k	Time at which iteration k took place.
$\mathbf{\Delta}(k)$	Difference between 2 consecutive assignment matrices $\mathbf{X}(t_k) - \mathbf{X}(t_{k-1})$.
$Rscore_k$	Rebalance cost at iteration k based on $\mathbf{\Delta}(k)$.
$\mathbf{y}(t)$	Vector that indicates whether a consumer is in use at time t.
$s_j(t)$	Production rate of partition $j \in P$ at time t.
$\phi(\delta)$	Uniform random number generator in range $[-\delta, \delta]$.
δ	Variability parameter to compute $\phi(\delta)$.

Given a measurement of the partitions' writing rate at time instant t, the problem involves minimizing the operational cost (number of consumers) while guaranteeing that each partition is assigned to a single consumer. As such, we use the bin packing formulation (Eqs. (3) - (7)) with a time variable t, which accounts for the times at which the measurement and the new consumer group configuration are computed.

$$\min \quad \sum_{i \in B} y_i(t) \tag{3}$$

$$\text{subject to} \quad \sum_{j \in P} \frac{s_j(t)}{C} \cdot x_{ij}(t) \leq y_i(t), \quad \forall\, i \in B \tag{4}$$

$$\sum_{i \in B} x_{ij}(t) = 1, \quad \forall\, j \in P \tag{5}$$

$$y_i(t) \in \{0, 1\}, \quad \forall\, i \in B \tag{6}$$

$$x_{ij}(t) \in \{0, 1\}, \quad \forall\, i \in B, j \in P \tag{7}$$

Set B corresponds to the available consumers, and P is the list of partitions to be reassigned to consumers. Decision variables $x_{ij}(t)$ and $y_i(t)$ indicate whether a partition j is assigned to a consumer i and whether consumer i is used at time t. Additionally, $x_{ij}(t)$ forms a matrix $\mathbf{X}(t)^{|B|\times|P|}$ and $y_i(t)$ provides the column vector $\mathbf{y}(t)^{|B|\times 1}$. The production rate measurement of partition j at time t in $bytes/s$ is denoted by $s_j(t)$, and C denotes the maximum consumption rate of a consumer. For constraints (4) and (5), the former assures that the sum of the partitions' writing rates assigned to any consumer does not exceed its capacity, and the latter guarantees that each partition is assigned a consumer.

4.2. Rebalance cost

To date, we have considered only a bin packing formulation for a single iteration; nevertheless, we must adapt the formulation to account for the dynamic nature of the production rate in each queue, which varies over time t. As a result, we arrive at the variable item size bin packing formulation (VISBP).

The VISBP heuristic algorithm operates sequentially and at discrete times instants. We use $k, k \in \mathbb{N}$, to denote the sequential iterations at which a new consumer group configuration is computed. Additionally, t_k represents the iteration execution time. It is also important to note that between any two consecutive iterations k and $k-1$, $t_k - t_{k-1}$ is variable. This occurs because a new partition for consumer assignment is computed depending on a set of conditions, e.g., whether a single consumer has exceeded capacity or a certain amount of time has elapsed since the last time a reconfiguration was evaluated. This approach is further described for the state machine presented in Section 7.3.

At any time $t < t_1$, the consumer group is not yet assigned any partitions and therefore, the assignment matrix of size $|B| \times |P|$ is empty:

$$\mathbf{X}(t) = \begin{bmatrix} 0 & \dots & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & \dots & 0 \end{bmatrix}, \quad \forall\, t < t_1 \tag{8}$$

Additionally, for any given $t_k, k > 1$, a partition–consumer assignment is computed at t_{k-1}, $\mathbf{X}(t_{k-1})$. Then, we compute $\mathbf{\Delta}(k)$ to determine which partitions have to be reassigned to different consumers via Equation (9).

$$\mathbf{\Delta}(k) = \mathbf{X}(t_k) - \mathbf{X}(t_{k-1}), \quad \forall\, k > 1 \tag{9}$$

For a cell $\Delta_{ij}(k)$ in $\mathbf{\Delta}(k)$, if its value is -1, it indicates that consumer i should stop consuming from partition j. If $\Delta_{ij}(k) = 1$, then consumer i should start consuming from partition j. Finally, if the value is 0, then no change is made to consumer i' assignment with regards to partition j.

Given that $\mathbf{\Delta}(k)$ encodes which partitions have to be reassigned from one consumer to another, there is an additional cost to consider related to reassigning a partition. Since only one consumer can read from a partition at a given time, when a partition needs to be reassigned, the consumer that is currently reading from that partition must first stop consuming to allow the new consumer to start reading data from it. Owing to this process, there is a time interval during which data are not consumed from the partition being reassigned, which leads to increasing consumption latency for the messages produced into this queue.

Our second objective is to minimize the latency between the time a message is produced and the time it is consumed. However, evaluating the latency for a set of messages given consumer group reconfiguration is nontrivial since latency can be measured only after a message has been consumed and not at the time of reconfiguration. For this reason, we propose the Rscore metric as a proxy for the latency cost. This metric considers the cost of consumer group reconfiguration.

A partition is reassigned at iteration k if its column in $\mathbf{\Delta(k)}$ contains both a 1 and a -1. When determining whether partition j must be reassigned, $\mathbf{\Delta_j(k)}$ denotes the j'th column of the matrix, which in turn is the reassignment vector for partition j at iteration k. The column vector in Equation (10) indicates that consumer $|B|$ should stop reading data from partition j so that consumer 1 can start. Notably, due to the bin packing formulation in Equation (3) - (7), $\mathbf{\Delta_j(k)}$ can only have nonzero values in at most 2 rows (2 consumers).

$$\mathbf{\Delta_j(k)} = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ -1 \end{bmatrix} \tag{10}$$

When the column vector has only one nonzero value and the value is -1, then the consumer group must stop reading data from this partition at t_k (item departure). Otherwise, if the value is 1, the consumer group has not yet assigned this partition, and it is to start retrieving messages from this partition at t_k (item arrival).

Message latency increases if a reassignment of partitions to consumers is required, which we refer to as a rebalancing operation. The proposed Rscore metric, shown in Equation (11), is an inference of the latency at iteration k.

$$Rscore_k = \frac{1}{C} \sum_{j \in P_k} s_j(t_k), \quad \forall\, k \geq 1 \tag{11}$$

where P_k is the set of partitions to rebalance at time t_k, $s_j(t)$ is the writing rate of partition j at time t_k (bytes/s), and C is the maximum consumer reading capacity (bytes/s).

As presented in [32], the rebalancing cost can be related to the number of migrations performed, or it can be computed based on the total size of the items being migrated. The R score considers the latter measure, as it sums the total production rate of all partitions being reassigned.

In essence, the time required to rebalance all partitions, combined with the Rscore, determines the amount of time a dedicated consumer requires to process the data that accumulated while migrating partitions.

5. Proposed algorithm

The motivation for modifying the existing any fit algorithms lies in their focus on only reducing the number of bins used to pack a set of items, disregarding the cost associated with reassigning items. Any fit represents, in this case, either the best-fit or worst-fit assignment strategies. Algorithm 1 outlines the implementation of the modified any fit algorithm, which will be further described in the remainder of this section. This algorithm defines the procedure for assigning partitions to consumers at time t_k in iteration k.

Algorithm 1 Modified Any Fit Algorithm.

Input: CG (current consumer group configuration) &
U (currently unassigned partitions) &
`assignStrategy` (Best or Worst Fit) &
`sortStrategy` (cumulative or max partition sort)
Output: N (new consumer group configuration)

```
1:  N ← new ConsumerList with assignStrategy
2:  S ← sort CG with sortStrategy
3:  for c ∈ S do
4:      pset ← partitions assigned to c
5:      pset ← sort pset in decreasing order
6:      for i ← pset.size() − 1 to 0 do
7:          p ← pset[i]
8:          result ← N.assignOpenBin(p)
9:          if result = false then
10:             break
11:         end if
12:         pset.remove(p)
13:     end for
14:     if pset.size() = 0 then
15:         continue
16:     end if
17:     N.createConsumer(c)
18:     for p ∈ pset do
19:         result ← N.assign(c, p)
20:         if result = false then
21:             break
22:         end if
23:         pset.remove(p)
24:     end for
25:     U.extend(pset)
26: end for
27: U ← sort U decreasing order
28: for p ∈ U do
29:     N.assignBin(p)
30: end for
31: return N
```

The new assignment is initially empty (line 1) and is set to apply the assigning strategy passed as a parameter (best or worst fit). In the first iteration, no previous assignment exists, and at line 2, S is empty, as CG is also empty. Therefore, the program skips to line 27, where U contains all unassigned partitions. An offline bin packing any fit heuristic is applied from lines 28 to 30 (best or worst fit). As required by AF algorithms, items are ordered first in decreasing order at line 27.

From the second iteration onwards, the approach changes to a semi-online AF version. Here, the input list is divided into a set of batches, and each batch is packed with an offline AF heuristic [12]. Each batch represents each consumer from the previous assignment. In line 2, the list of consumers is ordered according to the sorting strategy (parameter *sortStrategy*).

From lines 3 to 26, the partitions contained in one batch (current consumer c) are packed. At line 5, the items are ordered in decreasing

Table 2
Modified implementations of the any fit algorithms.

Algorithm	Consumer Sorting Strategy
Modified Worst Fit (MWF)	Cumulative write speed
Modified Best Fit (MBF)	Cumulative write speed
Modified Worst Fit Partition (MWFP)	Max partition write speed
Modified Best Fit Partition (MBFP)	Max partition write speed

order. Then, from lines 6 to 13, starting from the smallest item, an attempt is made to pack as many items as possible in the current open bins. Prioritizing smaller items for assignment helps migrate partitions with the lowest production rates, which in turn reduces the rebalancing cost ($Rscore_k$). When all the items are assigned or the process fails in line 9, execution proceeds to line 14. At this point, if all items are packed, the iteration ends, and the program returns to line 3 to process the next batch. Otherwise, at line 17, a new consumer is created in N, becoming the current consumer responsible for handling the partition. From lines 18 to 24, the partitions are assigned in decreasing order to the same consumer to which they are currently assigned. This consumer can also be represented by the index of the row in the column vector $\mathbf{X}_j(t_{k-1})$, which presents the value 1. The largest partitions are packed first, so their production rates do not contribute to the rebalancing cost of this iteration. In this loop, if the process fails at line 20, the consumer cannot accommodate the remaining items, and the process proceeds to line 25, where those items are added to the unsigned list U. After all batches are processed, the execution resumes at line 27, and U contains all items that were not packed when processing each batch. These items are then packed from lines 28 to 30, as in the first iteration, which implies the creation of new consumers in comparison to the previous assignment.

Note that consumers are released if, in the loop from lines 6 to 13, all the items belonging to the current consumer are packed in the available bins. Then, the consumer is not created in the new assignment at line 17.

The proposed algorithms, which aims to improve the existing bin packing heuristics with respect to the rebalancing cost, are derived from all possible combinations of the aforementioned sort-and-fit strategies. These combinations are summarized in Table 2. The acronyms provided in the table are used in the Experimentation section to identify the algorithms.

With respect to the consumer sorting strategies, the max partition sort aims to improve the algorithm's performance with respect to the operational cost. This strategy assigns partitions in an approximately decreasing order, similar to the any fit decreasing algorithms, considerably improving the algorithm's asymptotic performance. On the other hand, the cumulative sort targets the rebalancing cost by ensuring that the partitions assigned to the largest consumer are handled first. As such, considering that the initial stage has few or no available consumers, the partitions are more often than not assigned to the same consumer, i.e., no rebalancing. In later iterations, considering that the algorithm has more open bins (consumers) and is handling smaller partitions, it will rebalance partitions more often, therefore aiming to minimize the operational cost. The algorithms' performance with respect to rebalancing and operational costs are shown in Section 6.4.

Concerning the asymptotic performance ratio (R_A^∞) of the proposed algorithm, it can be determined that offline and semi-online AF algorithms, which have known R_A^∞, are employed. The offline best fit decrease $R_{BFD}^\infty \approx 1.222$, and the worst fit decrease $R_{WFD}^\infty \approx 1.6910$ [21]. For semi-online AF approximation algorithm applied when rebalancing partitions, $1.22 \leq R_A^\infty \leq 1.25$ for any algorithm $A \in AF$ operating on a list presorted in decreasing order [38]. Therefore, when the worst fit is used, $R_{A1}^\infty \approx 1.6910$, and when using the best fit, $R_{A1}^\infty \approx 1.25$, where $A1$ the proposed algorithm. With respect to Algorithm 1's time complexity, the complexity of sorting the set of consumers is $O(B\,log\,B)$, and that for sorting the set of partitions is $O(P\,log\,P)$. The complexity of packing an item is $O(log\,P)$, but that for packing P times is $O(P\,log\,P)$. Given the restriction that there will never be more consumers than partitions, i.e., $P \geq B$, the algorithm's time complexity is $O(P\,log\,P)$.

6. Experimentation

In this section, we present a comparison, through simulation, of the proposed algorithms and the state-of-the-art algorithms adapted for running successive assignment iterations.

6.1. Approximation algorithm adaptation

The existing bin packing heuristic algorithms presented in Section 2 were developed to solve a single iteration of the bin packing problem. When considering $t_k, k > 1$, there already exists a consumer assignment partition given by $\mathbf{X}(t_{k-1})$.

As the algorithm progresses by assigning partitions to consumers, the list of consumers currently in use is encoded in $\mathbf{y}(t_k)$. Additionally, as described in Section 2, the list of open bins are consumers who are already in use in iteration k ($y_i(t_k) = 1, i \in B$), which can be used to assign additional partitions. Some approximation algorithms limit the number of open bins, e.g., the NFD algorithm allows only one open bin, which is always the last bin created.

When these algorithms cannot partition j into one of the existing open bins, a new bin must be created. This new bin is commonly described as the lowest index bin that has not yet been used. Given the current iteration k, the function $f : \mathbb{N} \to B$, which is defined in Equation (12), represents the lowest indexed unused consumer.

$$f(k) = \min\{i \in B \mid y_i(t_k) = 0\} \tag{12}$$

There is no guarantee that the bin returned by $f(k)$ is the current bin assigned to partition j resulting from the consumer group computation at t_{k-1}. Assigning the partition to the lowest index unused bin obtained from Equation (12) often leads to a partition being rebalanced, as highlighted in [33].

For this reason, we modify the decision of which bin to create when an item does not fit any of the existing open bins. To do so, we define $g : P \times \mathbb{N}_{\setminus\{1\}} \to B$ in Equation (13). This function receives as input a tuple $(j,k), j \in P \wedge k \in \mathbb{N}_{\setminus\{1\}}$, where j is a partition and k is the current iteration. The consumer assigned partition j from iteration $k-1$ is output.

$$g(j,k) = i, \text{ such that } i \in B \ \wedge \ x_{ij}(t_{k-1}) = 1. \tag{13}$$

As such, when a new bin needs to be created for these algorithms, we first retrieve the consumer that is currently assigned the partition from t_{k-1}. If the consumer is currently not in use, i.e., $y_{g(j,k)}(t_k) = 0$, then consumer $g(j,k)$ is selected. Otherwise, the lowest indexed unused bin (Equation (12)) is used. This procedure is described by Equation (14), where $h(j,k)$ determines the consumer to be used for partition j. This adaptation increases the likelihood of the partition being assigned the same consumer, which minimizes the rebalancing cost without compromising the operational cost.

$$h(j,k) = \begin{cases} g(j,k), & \text{if } y_{g(j,k)}(t_k) = 0 \ \wedge \ k > 1 \\ f(k), & \text{otherwise} \end{cases} \tag{14}$$

6.2. Test data generation

To compare the performance of approximation algorithms, it is common to evaluate them with respect to their average-case performance [30] and [33]. Similar to the evaluation procedure described in [30], we generate streams of production rate measurements, where, from one measurement to the next, the measured production may vary up to 25% of a consumer's consumption capacity.

Throughout this section, we use the term measurement to represent a map of values that indicate the speed for each partition of interest and

the term stream to represent a sequence of measurements in the form of a list. Each measurement of a stream simulates item sizes for an instance of the BP problem, which is to be fed as input to the approximation algorithms.

The production rate of each partition j, $\forall j \in P$ is given by Equation (15) at iteration t_k over N iterations.

$$s_j(t_k) = \max\{0, \min\{s_j(t_{k-1}) + \frac{\phi(\delta)}{100} \cdot C, C\}\}, \quad \forall\, j \in P \ \wedge\ k \in \{1, 2, ..., N\} \tag{15}$$

where $\phi(\delta)$ is a uniform random function that selects a value between $[-\delta, \delta]$. Equation (15) requires defining the initial conditions $s_j(t_1)$, $\forall j \in P$. We tested four different approaches for the partitions' initial value: choosing a random value between $[0, 100]\% \cdot C$; setting the initial speed of all partitions to 0; setting the initial speed of all partitions to $50\% \cdot C$; and setting the initial speed of all partitions to C. Given that there was no significant difference in the outcome of these variations, the presented results were obtained using the streams of data generated with an initial speed for all partitions that was randomly selected between $[0, 100]\% \cdot C$.

6.3. Performance metrics

The goal of this section is to present a way to test different algorithms under the same conditions to compare their performance concerning operational and rebalancing costs. On the basis of real measures of a Kafka environment, shown in Section 8, throughout this section, we consider the following assumptions: an iteration has a duration of $30s$; a new measurement is read every 30 s; the production rate is stable throughout a whole iteration k; and it takes 5 s to rebalance partitions.

We consider latency as the main performance metric and evaluate it as the time elapsed between the moment the data are made available for consumption in a given queue and the moment a consumer reads the data. Under stable conditions, there is no latency; however, as the production rate in each partition varies, it may be necessary to rebalance partitions from one consumer to another.

There are two cases that can induce latency in a consumer group management solution. First, considering the set of partitions that remain assigned to the same consumer between iterations (fixed partitions), if the sum of their production rate exceeds the consumer's maximum capacity, then the latency will start increasing. Second, partitions that are reassigned to another consumer (rebalanced partitions) do not have their data consumed during the rebalancing phase. As such, the rebalanced partitions will experience increased latency soon after the rebalancing procedure is terminated. Consequently, in this section, we discuss the procedure used to evaluate the latency for both fixed and rebalanced partitions.

The sum of the rates at which data are read from the fixed and rebalanced partitions may never exceed the consumer's maximum consumption rate $\bar{C}$. Nevertheless, in our bin packing formulations (Section 5), we refer to capacity C, which represents the maximum total production rate that can be assigned to a single consumer. The excess capacity expressed by $\bar{C} - C$ is convenient for the bin packing algorithms to reduce the latency induced by their rebalancing procedures. Conversely, if there is no excess capacity, a consumer may increase its latency indefinitely. The capacity C is determined on the basis of an iteration's time window (T_W) and the rebalancing time measured for a particular system (T_R), $C = \frac{T_W - T_R}{T_W}\bar{C}$. For our experiment, with $T_W = 30$ and $T_R = 5$, $C = \frac{5}{6}\bar{C}$.

The set of partitions rebalanced to consumer c in iteration k is computed via Equation (16).

$$P_R^k(c) = \{j \mid \Delta_{cj}(k) = 1 \ \wedge\ j \in P\} \tag{16}$$

The condition $\Delta_{cj}(k) = 1$ ensures that a partition is assigned to consumer c at iteration k but is not at $k-1$ (Section 4).

On the other hand, Equation (17) specifies the partitions that are carried over (fixed partitions) from the previous iteration.

$$P_F^k(c) = \{j \mid X_{cj}(k) = 1 \ \wedge\ j \notin P_R^k\ \forall\, j \in P\}, \tag{17}$$

where $X_{cj}(k) = 1$, which ensures that the partition is assigned to consumer c in iteration k. Additionally, this condition enforces that j does not belong to the set of rebalanced partitions computed by Equation (16).

The consumer's fixed partitions contain the data produced in partitions $P_F^k(c)$, while the rebalanced partitions contain the data produced in partitions $P_R^k(c)$. The rate at which data are produced into the rebalanced and fixed partitions is computed as the sum of each partition's write rate. Hence, the rate of production in the fixed queue is expressed by Equation (18), and the rate at which data are produced in the rebalanced queue is expressed by Equation (19).

$$W_F^k(c) = \sum_{j \in P_F^k(c)} s_j(t_k) \tag{18}$$

$$W_R^k(c) = \sum_{j \in P_R^k(c)} s_j(t_k) \tag{19}$$

With respect to the rate at which the consumer reads data from each queue, if the write rate in the rebalanced queue $W_R^k(c) = 0$, then the speed at which the consumer reads data from the fixed queue is $R_F^k(c) = \bar{C}$; otherwise, $R_F^k(c) = min(\bar{C}, W_F^k(c))$. That is, the consumer reads from the fixed queue with its full capacity if there are no rebalanced partitions or if the rate of its consumption perfectly matches the rate of production into this queue (limited to its capacity). If the consumer has newly assigned partitions (rebalancing), which is facilitated by our bin packing algorithms, all excess consumption throughput is dedicated to the rebalancing queue; therefore, the rate at which the consumer reads from the rebalancing queue is $R_R^k(c) = \bar{C} - R_F^k(c)$. The reason that only the excess throughput is allocated to rebalanced partitions is because a partition's consumption rate is a noncontrollable consumer parameter; therefore, in a worst-case analysis, we have to assume that only the spare throughput is used to recover from the rebalanced partitions' latencies.

The latency experienced in a consumer's c fixed queue is evaluated by Equation (21) for a given byte i, which expresses the latency as a linear equation whose rate depends on the queue's consumption and production rates. The intercept of the equation reflects the consumer's starting point at iteration k, which depends on the measured latency of the last byte produced at iteration $k-1$. As such, let $T_F^k = 30 \cdot W_F^k(c)$ define the total amount of data produced at iteration k, where 30 is our iteration time window. We can now define the intercept through Equation (20), which also considers the consumer's first iteration ($k = 1$), where $b_F^k(c) = 0$.

$$b_F^k(c) = \begin{cases} L_F^{k-1}(c, T_F^{k-1}(c)), & if\ k > 1 \\ 0, & otherwise \end{cases} \tag{20}$$

Finally, to compute a consumer's c rebalanced queue latency for byte i, we use Equation (22). This latency is expressed by a linear equation, whose rate of change depends on its consumption and production rates. However, this equation's intercept is a constant value and represents the time to complete the rebalancing procedure, which in our experimental setup is 5 s.

$$L_F^k(c, i) = max\left\{\left(\frac{1}{R_F^k(c)} - \frac{1}{W_F^k(c)}\right) i + b_F^k(c), 0\right\}\ \forall i \in \mathbb{N}_{\leq T_F^k(c)} \tag{21}$$

$$L_R^k(c, i) = max\left\{\left(\frac{1}{R_R^k(c)} - \frac{1}{W_R^k(c)}\right) i + 5, 0\right\},\ \forall i \in \mathbb{N}_{\leq T_F^k(c)} \tag{22}$$

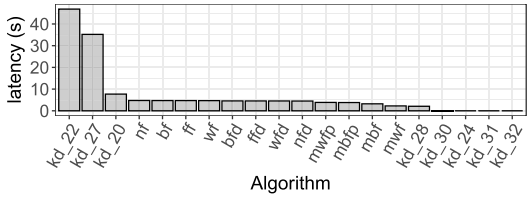


Fig. 2. 99th percentile latency for the top 20 performing algorithms.

6.4. Results

To evaluate each algorithm under the same conditions, we first generate a stream of measurements through the procedure described in Section 6.2, considering 32 partitions, $N = 100$ measurements and $\delta = 5$. For reproducibility, the data stream and the source code are available in a repository.[3] We assume that each measurement read from the stream represents a stable production rate to each partition for the next 30 s. As a result, each experiment has a duration of 50 minutes, and the production rate into a partition may vary up to 100 kB/s between two consecutive measurements. On average, the total data produced per partition is 2.5 GB, and the average production rate is 830 kB/s. This represents significant amounts of data, variability, and simulation time. For comparison, [8] conducted a 2-hour experiment, producing a total of 1.35 M events, with an estimated 1.4 kB per message (1.9 GB total data produced), 378 MB of data per partition (5 partitions), an average production rate of 105 kB/s per partition, and production rate variations between 23 and 42 kB/s.

We then create consumer group configurations on the basis of each algorithm's assignment strategy and compute the latency per the models defined in Equations (21) and (22). These experiments are run for Kafka configurations with 1, 3, 5, 10, 11, ..., 32 consumers, for our modified any fit algorithms and the any fit algorithms selected for comparison in Sections 2 and 3. This evaluation provides a direct comparison between our proposed approach from Section 5, solutions that leverage Kafka's assignment strategy, such as KEDA, and other bin packing algorithms presented in the literature.

Concerning operational costs, which is measured as the number of active consumers, the Kafka algorithms (`kd_1`, `kd_2`, `kd_3`, ..., `kd_32`) do not resize the number of consumer throughout the experiment. Therefore, their average operational cost is equal to the integer value in their name, e.g., `kd_25` operates with 25 consumers. Our proposed MWF algorithm uses an average of 15 consumers throughout the whole experiment.

In terms of latency, Fig. 2 presents plots the top 20 performing algorithms on the basis of the 99th percentile latency. As shown, the MWF algorithm is outperformed by only the Kafka configurations with 24, 28, 30, 31 and 32 consumers. The MWF 99th percentile latency is 2.24 s, whereas the aforementioned Kafka configurations achieve 99th percentile latencies of 0 s, 2.09 s, 0.23 s, 0 s and 0 s, respectively. On the other hand, these same Kafka configurations represent increased operational costs of 60%, 86.7%, 100%, 106.67% and 113.33%, respectively, compared with that of the MWF algorithm's, which uses an average of 15 consumers. Finally, Kafka's configuration with the same operational cost, `kd_15`, yields a 99th percentile latency of 364.66 s, which is 168× greater than the latency observed with the MWF algorithms.

Counterintuitively, with Kafka's assignment strategy, increasing the number of consumers in a group does not reliably improve the group's performance with respect to latency, as shown in Fig. 2. This outcome is a consequence of Kafka attempting to assign an equal number of partitions to each consumer without considering the total production rate assigned to each consumer, which, more often than not, exceeds the consumer's capacity. In other words, simply increasing the number of consumers and leveraging Kafka's assign strategy, as is the case for the KEDA's autoscaling solution, is an unreliable method for improving the group's latency performance.

[3] https://github.com/landaudiogo/kafka-consumer-group-autoscaler.

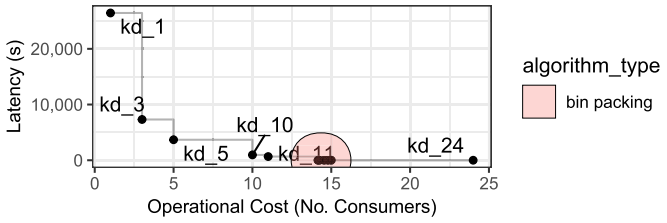


Fig. 3. Pareto front for the 99th percentile latency and operational cost.

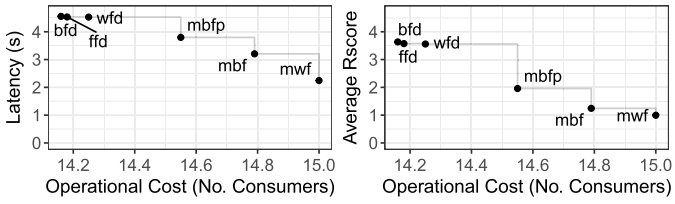


Fig. 4. Pareto front for the modified fit algorithms. The figure on the left presents a comparison of the 99th percentile latency with the operational cost, and the figure on the right shows a comparison the average R score with the operational cost.

Finally, we identify the Pareto front to determine the most efficient set of solutions, provided that there are trade-offs within a multiobjective optimization problem. As shown in Fig. 3, the Kafka configurations that are a part of the Pareto front are configurations with 1, 3, 5, 10, 11 and 24 consumers. Fig. 4 presents a zoomed-in view of the bin packing algorithms highlighted in red in Fig. 3. Excluding MWFP, the modified algorithms are consistently a part of the Pareto front, as shown in Fig. 4 (left). This finding suggests that algorithms are competitive alternatives when solving this variation of the BP problem. Additionally, the proposed algorithms perform better than the state-of-the-art algorithms do, as MWF achieves a 48% reduction in latency, with only a 6% increase in operational costs compared with those of BFD. The Pareto front also informs us of which algorithm to select on the basis of our optimization targets. On the one hand, if we prioritize the operational cost, BFD is the best algorithm. On the other hand, if our goal is to minimize the latency, the best performing algorithm is MWF. However, when attempting to balance both optimization goals, we should select MBFP when we assign a higher priority to the operational cost and the MBF when prioritizing latency.

Moreover, Fig. 4 shows how the Rscore is a suitable proxy for the latency cost. This is supported by the observation that the average R score ranks the any fit algorithms in the same order as their ranking on the basis of the 99th percentile latency.

7. Consumer group autoscaler implementation

In this section, we consider Kafka as the message broker for evaluation purposes, although the same methodology can be applied to other message broker systems. There are three components, presented in Fig. 5, that interact with one another to model the BP problem. This design supports a fully dynamic pipeline capable of autoscaling a consumer group on the basis of the current load of data produced in the partitions of interest. Although our analysis focuses on implementation within a single consumer group, this approach is applicable to a multiconsumer group environment, whereby each group elects a controller, and this component manages the partition–consumer assignment. This process is similar to Kafka's default process of managing consumer groups.

The monitoring process measures the write speed of each partition, which is equivalent to specifying the size of the items of the bin packing problem. This information is then sent to the controller, which is respon-

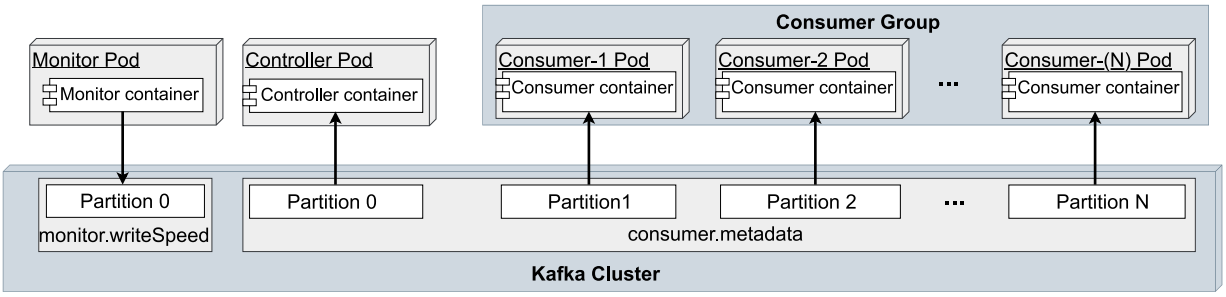


Fig. 5. System architecture based on Kafka, consisting of monitoring, controller and consumer processes.

sible for managing a consumer group (creating and deleting consumer instances) and mapping each partition to a consumer. The consumers are then informed of their tasks and read the data from the partitions that were assigned to them by the controller.

This system was applied in a real-world setting, with the goal of extracting data published to Kafka topics for insertion into a data lake. The source code is available in the repository mentioned above.

7.1. Monitoring process

To solve the BP problem, initially, the controller requires as input the write speed of each partition from which the consumer group is interested in consuming data. The monitoring process is responsible for providing this information.

Kafka provides an Admin client that enables cluster administration and querying. This client or class includes the `describeLogDirs()` method, which queries the cluster to retrieve the number of bytes in each TopicPartition. A TopicPartition is a string–integer pair that identifies any partition (integer) within a topic (string).

Each time the partition size is queried by the admin client, a timestamp is attached to the measurement, and it is added to the end of a queue. Any query that is older than 30 s, which will always be at the front of the queue, is removed. To determine the write speed of a single partition, the last and first elements in the queue, representing the most recent and oldest measurement of the partition size within the last 30 s, are used to compute the ratio between the difference in bytes and the difference in time ($bytes/s$). This value also represents the average write speed over the last 30 s time window. The smaller the time window is, the better the measurement reflects the producer's network capacity as opposed to measuring the current rate of production in a partition. Additionally, with a small time window, 2 consecutive events arriving close to the same time also lead to misleading and noisier production rate measurements. After evaluating several windows of different sizes, 30 seconds provides an acceptable trade-off between the amount of noise and the time required for the measurement to converge to a stable production rate.

After computing the write speed for all the partitions of interest, the information must be transmitted to the controller. To benefit from an asynchronous approach, this monitoring process communicates with the controller process via the Kafka topic illustrated in Fig. 5 as `monitor.writeSpeed`.

7.2. Consumer process

The consumer progresses through four important phases in its operation to approximate its consumption rate to a constant value when operating at its peak performance. These phases repeat cyclically until the consumer receives an external termination signal. Fig. 6 provides a visual representation of the consumer's process.

The consumer is configured with two important parameters, BATCH_SIZE and WAIT_TIME_SECS, which indicate, the number of bytes the consumer waits to gather information a single iteration and the maximum amount of time it is allowed to wait before gathering the information.

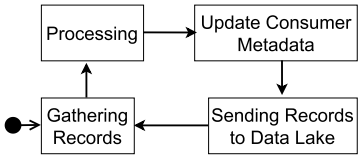


Fig. 6. Consumer insert cycle.

In the first phase, the consumer attempts to retrieve BATCH_SIZE bytes from the partitions it is consuming. Once this condition is satisfied or WAIT_TIME_SECS is exceeded, the consumer moves on to the second phase. Here, the consumer deserializes and processes each individual record and batches records that originated from the same topic together.

In the third phase, the consumer sends its records to the data lake, performing an asynchronous request for each topic from which it retrieves data in the same iteration. After forwarding all records to their designated tables in the data lake, the consumer then verifies its metadata queue to identify any changes in the state messages to be consumed. This queue serves as the mechanism through which the controller process (Section 7.3) informs each consumer of its assignments. If new messages are available, the consumer reads all the messages in the queue and updates its state. Only after successfully updating its state and persistent metadata does the consumer send an acknowledgment back to the controller to indicate a successful change in state. This cycle repeats until the controller removes the consumer from the active consumer group.

7.3. Controller process

The controller is the component of the system that is responsible for orchestrating and managing the consumer group. During the monitoring process, the write rate of each partition is computed and used as input for the approximation algorithm. This algorithm outputs a new consumer group configuration. On the basis of this configuration, the controller creates any missing consumers, communicates changes in assignment to each consumer in the group, and deletes the consumers who are not required in the new computed group's state.

As shown in Fig. 5, two topics serve as communication links between the three system components. The topic `monitor.writeSpeed` is used by the controller to read partition write speeds, while `consumer.metadata` is used to send change messages to consumers. In terms of the latter Kafka topic, partition 0 is reserved for communication directed toward the controller, whereas the remaining partitions within `consumer.metadata` represent a one-to-one mapping to each consumer. Therefore, if the controller needs to communicate with consumer c, it sends a record to partition c.

This communication architecture is designed to achieve an efficient communication model, defined as the amount of information that is relevant relative to the amount of data read. With this pattern, all the data read is relevant to the reading entity.

The controller can be summarized by a state machine intended to continuously manage a group of consumers and their assignments, as illustrated in Fig. 7.

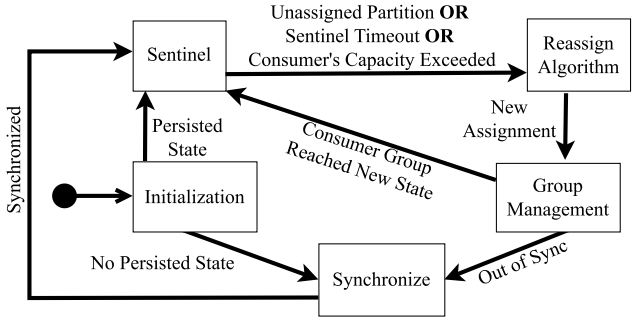


Fig. 7. Controller state machine.

The Sentinel state has the controller read the information published by the monitoring component to update its current consumer group's state and determine whether it has to recompute the group's assignments. If one of the exit conditions is determined to be true, the controller transitions into the reassign algorithm state. At this stage, one of the approximation algorithms is employed to heuristically solve the bin packing problem.

The output of the approximation algorithm indicates the desired state in which the controller intends to configure its consumer group. As such, the first step in the group management state involves calculating the difference between the current state of the group and the desired state. This difference conveys information regarding the following: the consumers to be created, the partitions each consumer has to stop consuming from, the partition each consumer has to start consuming from, and finally, the consumers that have no assignment and therefore can be decommissioned. First, the controller creates new consumer instances, which involves the controller communicating with a Kubernetes cluster to create deployments related to each consumer. We chose to have different deployments for each consumer so that the information related to which `consumer.metadata` partition the consumer reads from could be transmitted through the deployment's manifest `metadata.name` attribute. Second, the controller informs each consumer of their change in state by transmitting the start- and stop-consuming messages to each consumer via their respective `consumer.metadata` partition. Importantly, communicating the change in state is a synchronous process to ensure that, while rebalancing partitions, there is no more than one consumer from the group reading data from a partition. Therefore, for each partition being rebalanced, the controller first sends out the stop-consuming message, and only after the consumer informs the controller of having acted upon the message can the controller send out the start-consuming record to the new consumer. After consumers have reached their intended state, the controller shuts down the inactive consumers.

The synchronized state ensures alignment between the consumer group's real state with the controller's perceived state. This mechanism allows the controller is capable of recovering from an unexpected termination, which can lead to an inconsistent perception of the consumer group's state.

8. Empirical evaluation

The consumer group autoscaler was developed as a proof of concept to demonstrate how this variation of the bin packing problem could be applied in a message broker environment. As such, we present the results obtained when running this system in a real-world production environment, where the message broker facilitates communication for an online order-fulfillment application, using the MWF algorithm for assignment. The computational resources utilized are summarized in Table 3, where AWS refers to Amazon Web Services and where GCP refers to the Google Cloud Platform.

One of the first assumptions was that this problem is a variation of the single bin size bin packing problem; therefore, the consumer should present a constant maximum capacity when challenged to work at peak performance. To validate the assumption, the consumer was tested in three very disparate conditions, specified in Table 4, all requiring the consumer to be reading the data at its maximum consumption rate. Between each test, we varied the average number of bytes present in each partition, the total number of bytes available, and the number of partitions assigned to the consumer instance. The data in Table 4 originated from an order-fulfillment application's production cluster currently in use to test our implementation. Fig. 8 presents the results of having the consumer run under these conditions and clearly shows a common mode for the maximum consumption rate, approximately 2.3 MB/s. We therefore consider this assumption to be valid under normal conditions.

Table 3
Infrastructure used for experimentation.

Component	Cloud Platform	Resource Classification
Kafka Cluster (3 Nodes)	AWS	r5.xlarge
Kubernetes	GCP	GKE autopilot
Monitor	GCP	Kubernetes Deployment (GKE)
Controller	GCP	Kubernetes Deployment (GKE)
Consumer Group	GCP	As many Deployments as consumers in the group (GKE)

Table 4
Testing conditions to obtain the consumer maximum throughput measure.

Test ID	Total Bytes	Average Bytes per Partition	Number of Partitions
Test 1	648 $Mbytes$	20 $Mbytes$	32
Test 2	100 $Mbytes$	0.86 $Mbytes$	116
Test 3	678 $Mbytes$	4 $Mbytes$	144

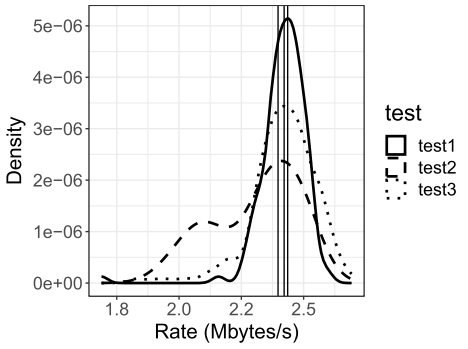


Fig. 8. Density plot for the consumer's measured throughput for the three tests conditions.

With the goal of testing the system as a whole, we statistically summarize the time required the autoscaler to progress through each event required to dynamically scale the group of consumers.

To fully evaluate the system's response to high load variations and increase reproducibility, the monitor component was replaced by the measurement streams described in Section 6.2, with $N = 500$ and $\delta = 25$. As a result, the implementation was tested for a total duration of 4 hours and 10 minutes, with a 500 kB/s maximum partition production rate variation between two consecutive measurements, i.e., a production rate variation of up to one fourth of the consumer throughput every 30 seconds.

The system's response time starts with the monitoring process measuring the current production rate of each partition. The measurement performed by the monitor component from Section 7.1 is a moving average with a time frame of 30 seconds. As such, this component would require 30 seconds to converge to a stable rate of production. This event is represented in Fig. 9 as τ_1.

The second event, τ_2, represents the time required by the controller to compute the consumer group's new assignment, i.e., computing the difference between the new and current states, and send an asyn-

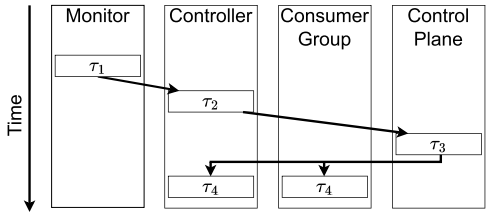


Fig. 9. System sequence of response events.

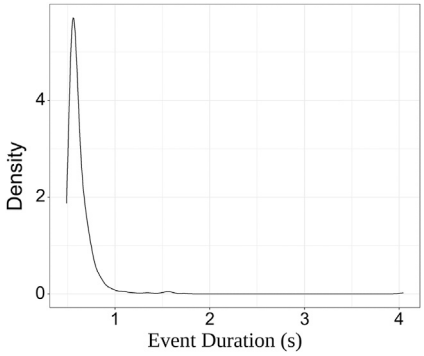


Fig. 10. Distribution of τ_2 for 1345 observations.

chronous request to the Google Kubernetes engine (GKE) cluster for every new consumer instance that is created.

The time required by the controller to complete this procedure depends on the number of partitions distributed between the consumer group, the algorithm the controller is executing to identify a new consumer group assignment, and the number of new consumer instances created in the GKE cluster.

For the tested input data, there were at most 32 partitions to rebalance, and at most 20 consumers needed to be created in a single iteration. Fig. 10 presents an approximation of the observed data points with a continuous probability density function (PDF) and is constructed such that the area under the curve for the full domain is equal to 1. As a result, the PDF indicates that the probability of τ_2 being less than 2 s is approximately 1 ($P(\tau_2 < 2s) \approx 1$). Given that the activity performed during τ_2 is dominated by the computation of a new consumer group assignment, as described in Section 5, its time complexity is $O(P\,log\,P)$, which depends on the number of partitions to be assigned.

After the asynchronous request is sent to the GKE cluster, the autoscaler enters τ_3, where it has to wait for the control plane to schedule the consumer pods to cluster nodes and run the containers. From Fig. 11, there are two main clusters of data points, which can be summarized into two different scenarios in which the GKE cluster can be located.

The first, and most frequent (88%), GKE cluster requires between $[10, 50]$ seconds for a new consumer instance to be ready. This case arises when the controller requests the creation of new consumer instances and the Kubernetes cluster has sufficient resources available. As such, the event's duration is related to the time required for the scheduled node(s) to download the image from the container registry and create the containers.

The second group of data points, any time span greater than 50 s, representing 12% of the data points, occurs when the Kubernetes cluster does not have any available resources. Here, the actions the cluster undergoes add a new node to the cluster, and only then is the consumer instance scheduled into the new available node. Owing to the autoscaling feature of the GKE cluster, this process is performed automatically but it is also more inconsistent, with data points requiring up to 500 s, although very sporadically. To mitigate the fact that there is no control over time, the Kubernetes cluster schedules and starts the bins. Thus, techniques that reduce the size of the image that has to be downloaded by the nodes and the retrieval time [39] can be applied. Additionally,

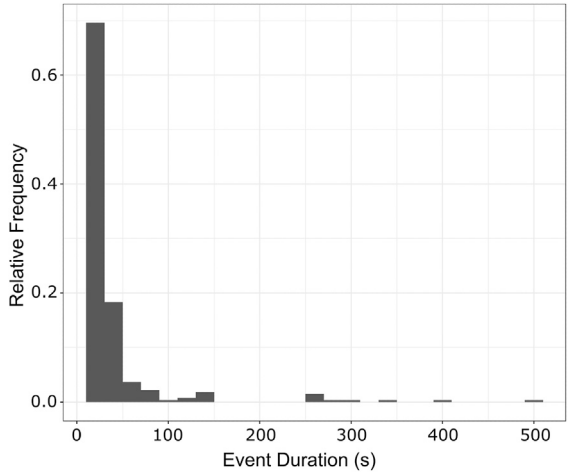


Fig. 11. Histogram of τ_3 for 273 observations.

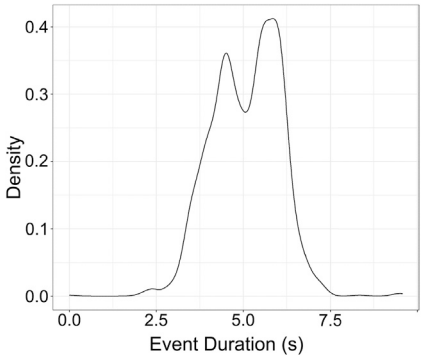


Fig. 12. Distribution of τ_4 for 1331 observations.

other strategies can be developed for minimizing τ_3, which are beyond the scope of this paper. For a fair comparison, which is immune to external factors, in our experiment, we did not consider τ_3 in the algorithm performance evaluation.

Finally, after the consumer instances run and are ready to receive assignments, the controller must communicate each consumer's assignment within the newly computed group configuration, which is represented by τ_4 in Fig. 12. Owing to the synchronous nature of rebalancing, i.e., a partition from one consumer to another, the controller must first send out a stop-consuming message for each partition that has to be rebalanced, wait for the consumer to respond, send out a start-consuming message and wait for a response. Without considering any processing and network delays, at worst, the controller might have to wait for two consumer cycles to rebalance a partition.

A consumer's cycle consists of the different phases mentioned in Section 7.2. Only after performing its tasks does the consumer verify the metadata queue to check if it has received any change in state messages. For this reason, between two metadata reads from the `consumer.metadata` topic, one whole consumer cycle is required. After setting BATCH_SIZE to 5 Mbytes and WAIT_TIME_SECS to 1 s. The results in Fig. 8 indicate that the consumer has a maximum consumption rate of approximately 2.3 Mbytes/s, each consumer cycle requires approximately $2.5s$. Since the controller may have to wait for two consumer cycles, changing the group's state could take approximately 5 seconds, as shown in Fig. 12.

Notably, the more consumers there are in the group, the higher the probability that the controller has to wait one whole cycle after sending out the stop command and another whole cycle after a start command, as the communication would have to be performed with more consumers.

With respect to system scalability, given the procedure described in Section 7.1, the computational complexity is $O(P)$, which increases lin-

early with the number of partitions it has to track and is independent of the current production rate into the system. Furthermore, the latency to query the partitions' size depends on which brokers they are located in. However, considering that the querying can be performed in parallel, the latency is dominated by the slowest response obtained by one of Kafka's brokers.

9. Conclusions

The study presented in this paper focused on achieving a deterministic approach to determine the number of consumers required to work in parallel to ensure that the rate of consumption from a set of partitions is not lower than their combined production rate. Furthermore, the algorithm simultaneously aims to reduce the operational cost, which is a function of the average number of consumers used over time.

As such, we modeled the problem as a variation of the bin packing formulation, where the item sizes and bin assignments may vary over time. In light of these variations, item assignments can be rebalanced (bin assignments can change), which led to the creation of the Rscore metric (Section 4.2) to evaluate the rebalancing cost for a particular bin packing iteration. In this work, we focused on the case where a consumer presents a homogeneous consumption rate, which was shown to be a valid assumption under our configuration. However, in scenarios where consumers have different resource availability, or even in cases where a consumer is experiencing interference, this assumption no longer holds, requiring a bin packing formulation with heterogeneous bin sizes (consumption rates). We plan to address this problem in our future work.

To address the conflicting objectives of minimizing both the number of consumers and the rebalancing cost, in Section 5, we proposed four new variations of the bin packing approximation algorithms. In Section 6.4, we compared our algorithms with state-of-the-art heuristics against workloads developed with extreme production rates and variability conditions in mind. However, the workloads did not follow any real-world complex workload patterns. Therefore, we defer this analysis to our future work. Nevertheless, their comparison showed that our methods are competitive alternatives when solving this multiobjective optimization problem. In summary, in a message broker environment with high variability, our approach presented a 99th percentile latency of 2.24 s compared with Kafka's 364.66 s for both methods using the same number of consumers. Conversely, to obtain a lower 99th percentile latency than our approach does, Kafka requires at least 60% more consumers than our methods require. Considering only the state-of-the-art bin packing algorithms, the proposed MWF performs better than BFD does, as it reduces latency by 48%, with an increase of only 6%, in operational costs.

Furthermore, in Section 7, we provided an implementation of the BP heuristics to the Kafka message broker environment, which automatically scales a group of consumers and manages its partition–consumer assignments. This system was tested in a real production environment and, as expected, was capable of providing a solution that scales a group of consumers on the basis of the system's load, as shown in Section 8.

Finally, in this paper, we challenged some of Kafka's core functionalities and provide tested alternatives that can be used as a foundation to improve the way in which a consumer's load is modeled and the manner in which the load (partitions) is distributed between the elements of a consumer group.

CRediT authorship contribution statement

Diogo Landau: Writing – original draft, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Nishant Saurabh:** Writing – review & editing, Validation, Methodology. **Xavier Andrade:** Writing – original draft, Methodology, Formal analysis. **Jorge G. Barbosa:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Code and data is available online and a link is provided in the manuscript.

References

[1] Luca Magnoni, Modern messaging for distributed systems, J. Phys. Conf. Ser. 608 (2015) 012038, IOP Publishing.
[2] John Vineet, Xia Liu, A survey of distributed message broker queues, arXiv preprint, arXiv:1704.00411, 2017.
[3] Sophie Blee-Goldman, From eager to smarter in apache kafka consumer rebalances, https://www.confluent.io/blog/cooperative-rebalancing-in-kafka-streams-consumer-ksqldb/, 2020.
[4] Paul Le Noac'h, Alexandru Costan, Luc Bougé, A performance evaluation of apache kafka in support of big data streaming applications, in: 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 4803–4806.
[5] N. Sanjana, Sinchana Raj, S. Sandhya, et al., Real-time event streaming for financial enterprise system with kafka, in: 2023 3rd Asian Conference on Innovation in Technology (ASIANCON), IEEE, 2023, pp. 1–6.
[6] Eldor Elad, Kafka Troubleshooting in Production, Springer Books, 2023.
[7] Jiwon Bang, Siwoon Son, Hajin Kim, Yang-Sae Moon, Mi-Jung Choi, Design and implementation of a load shedding engine for solving starvation problems in apache kafka, in: NOMS 2018-2018 IEEE/IFIP Network Operations and Management Symposium, IEEE, 2018, pp. 1–4.
[8] Mazen Ezzeddine, Françoise Baude, Fabrice Huet, Tail-latency aware and resource-efficient bin pack autoscaling for distributed event queues, in: CLOSER, 2024, pp. 50–64.
[9] Peeranut Chindanonda, Vladimir Podolskiy, Michael Gerndt, Self-adaptive data processing to improve slos for dynamic iot workloads, Computers 9 (1) (2020) 12.
[10] Mazen Ezzeddine, Gael Migliorini, Françoise Baude, Fabrice Huet, Cost-efficient and latency-aware event consuming in workload-skewed distributed event queues, in: Proceedings of the 2022 6th International Conference on Cloud and Big Data Computing, 2022, pp. 62–70.
[11] Giselle van Dongen, Dirk Van Den Poel, A performance analysis of fault recovery in stream processing frameworks, IEEE Access 9 (2021) 93745–93763.
[12] Edward G. Coffman, János Csirik, Gábor Galambos, Silvano Martello, Daniele Vigo, Bin packing approximation algorithms: survey and classification, in: Handbook of Combinatorial Optimization, 2013, pp. 455–531.
[13] Ori Hadary, Luke Marshall, Ishai Menache, Abhisek Pan, Esaias E. Greeff, David Dion, Star Dorminey, Shailesh Joshi, Yang Chen, Mark Russinovich, et al., Protean: vm allocation service at scale, in: Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation, 2020, pp. 845–861.
[14] Rina Panigrahy, Kunal Talwar, Lincoln Uyeda, Udi Wieder, Heuristics for vector bin packing, research. microsoft. com, 2011.
[15] Sebastian Berndt, Klaus Jansen, Kim-Manuel Klein, Fully dynamic bin packing revisited, Math. Program. 179 (1–2) (2020) 109–155.
[16] Krzysztof Fleszar, Khalil S. Hindi, New heuristics for one-dimensional bin-packing, Comput. Oper. Res. 29 (7) (2002) 821–839.
[17] Jatinder N.D. Gupta, Johnny C. Ho, A new heuristic algorithm for the one-dimensional bin-packing problem, Prod. Plan. Control 10 (6) (1999) 598–603.
[18] Pierre Hansen, Nenad Mladenović, An introduction to variable neighborhood search, in: Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization, Springer, 1999, pp. 433–458.
[19] Nenad Mladenović, Pierre Hansen, Variable neighborhood search, Comput. Oper. Res. 24 (11) (1997) 1097–1100.
[20] David S. Johnson, Alan Demers, Jeffrey D. Ullman, Michael R. Garey, Ronald L. Graham, Worst-case performance bounds for simple one-dimensional packing algorithms, SIAM J. Comput. 3 (4) (1974) 299–325.
[21] E.G. Coffman Jr., M.R. Garey, D.S. Johnson, Approximation algorithms for bin packing: a survey, in: Approximation Algorithms for NP-Hard Problems, 1996, pp. 46–93.
[22] Brenda S. Baker, Edward G. Coffman Jr., A tight asymptotic bound for next-fit-decreasing bin-packing, SIAM J. Algebraic Discrete Methods 2 (2) (1981) 147–152.
[23] Matteo Nardelli, Valeria Cardellini, Vincenzo Grassi, Francesco Lo Presti, Efficient operator placement for distributed data stream processing applications, IEEE Trans. Parallel Distrib. Syst. 30 (8) (2019) 1753–1767.
[24] Fabio Furini, Enrico Malaguti, Rosa Medina Durán, Alfredo Persiani, Paolo Toth, A column generation heuristic for the two-dimensional two-staged guillotine cutting stock problem with multiple stock size, Eur. J. Oper. Res. 218 (1) (2012) 251–260.
[25] Maxence Delorme, Manuel Iori, Silvano Martello, Logic based benders' decomposition for orthogonal stock cutting problems, Comput. Oper. Res. 78 (2017) 290–298.

[26] Mauro Dell'Amico, Fabio Furini, Manuel Iori, A branch-and-price algorithm for the temporal bin packing problem, Comput. Oper. Res. 114 (2020) 104825.
[27] Qiufen Xia, Weifa Liang, Wenzheng Xu, Throughput maximization for online request admissions in mobile cloudlets, in: 38th Annual IEEE Conference on Local Computer Networks, IEEE, 2013, pp. 589–596.
[28] Vincenzo Gulisano, Ricardo Jimenez-Peris, Marta Patino-Martinez, Claudio Soriente, Patrick Valduriez, Streamcloud: an elastic and scalable data streaming system, IEEE Trans. Parallel Distrib. Syst. 23 (12) (2012) 2351–2365.
[29] Thomas Heinze, Yuanzhen Ji, Yinying Pan, Franz Josef Grueneberger, Zbigniew Jerzak, Christof Fetzer, Elastic complex event processing under varying query load, in: BD3@ VLDB, vol. 1018, 2013, pp. 25–30.
[30] Shahin Kamali, Efficient bin packing algorithms for resource provisioning in the cloud, in: Algorithmic Aspects of Cloud Computing: First International Workshop, ALGOCLOUD 2015, Patras, Greece, September 14–15, 2015, in: Revised Selected Papers, Springer, 2016, pp. 84–98.
[31] Lei Shi, John Furlong, Runxin Wang, Empirical evaluation of vector bin packing algorithms for energy efficient data centers, in: 2013 IEEE Symposium on Computers and Communications (ISCC), IEEE, 2013, pp. 000009–000015.
[32] Zoltán Ádám Mann, Approximability of virtual machine allocation: much harder than bin packing, 2015.
[33] Weijia Song, Zhen Xiao, Qi Chen, Haipeng Luo, Adaptive resource provisioning for the cloud using online bin packing, IEEE Trans. Comput. 63 (11) (2013) 2647–2660.
[34] Supun Kamburugamuve, Geoffrey Fox, Survey of Distributed Stream Processing, Indiana University, Bloomington, 2016, p. 153.
[35] Khin Me Me Thein, Apache kafka: next generation distributed messaging system, Int. J. Sci. Eng. Technol. Res. 3 (47) (2014) 9478–9483.
[36] Supun Kamburugamuve, Leif Christiansen, Geoffrey Fox, A framework for real time processing of sensor data in the cloud, J. Sens. (2015) 2015.
[37] Yuanli Wang, Baiqing Lyu, Vasiliki Kalavri, The non-expert tax: quantifying the cost of auto-scaling in cloud-based data stream analytics, in: Proceedings of the International Workshop on Big Data in Emergent Distributed Environments, 2022, pp. 1–7.
[38] D.S. Johnson, Near-Optimal Bin Packing Algorithms, Doctoral dissertation, MIT, Cambridge, MA, 1973.
[39] Nishant Saurabh, Shajulin Benedict, Jorge G. Barbosa, Radu Prodan, Expelliarmus: semantic-centric virtual machine image management in iaas clouds, J. Parallel Distrib. Comput. 146 (2020) 107–121.



Diogo Landau is a PhD student at the Department of Information and Computing Sciences, Utrecht University, Netherlands. Previously he received his BSc and MSc degree in Electrical and Computer Engineering from the Faculty of Engineering of the University of Porto, Portugal. His research areas include distributed resource monitoring and provisioning, parallel computing, and systems performance modeling.



Nishant Saurabh is an assistant professor at the Department of Information and Computing Sciences, Utrecht University, Netherlands. Previously, he was a postdoctoral researcher at ITEC, University of Klagenfurt, Austria. He obtained his Ph.D. in 2021 from the University of Innsbruck, Austria. His research areas include resource management, performance modeling and optimization in distributed systems, Cloud and Edge computing.



Xavier Andrade received his MSc and PhD degrees in Industrial Engineering and Management from the Faculty of Engineering of the University of Porto (FEUP), Portugal. Currently, he lectures at multiple Portuguese institutions and builds academic-grade solutions for a number of e-commerce's decision problems. He is interested in Management Science and Operations Research.



Jorge G. Barbosa received his BSc degree in Electrical and Computer Engineering from Faculty of Engineering of the University of Porto (FEUP), Portugal; MSc in Digital Systems from University of Manchester Institute of Science and Technology, England, in 1993, and PhD in Electrical and Computer Engineering from FEUP, Portugal, in 2001. He is an associate professor at FEUP. His research interests include parallel and heterogeneous computing, scheduling in heterogeneous environments, and cloud computing.